Report: LED Brightness Control Using Arduino Uno and Potentiometer via PWM

Abstract

This project demonstrates dynamic, continuous voltage variation control over a Light Emitting Diode (LED) using an Arduino Uno R3 microcontroller platform. By leveraging the built-in Analog-to-Digital Converter (ADC) alongside Pulse Width Modulation (PWM), manual input from a 10 kOhm potentiometer is converted into a proportionate visual intensity output on the LED. Rather than relying on standard binary digital states (HIGH/LOW), continuous analog sensing maps real-time input to varying duty cycles. This document details the architectural hardware connections, mathematical mapping logic, algorithmic implementation, and hardware/simulation verification steps.

Objective

- To establish dynamic, analog-like control of LED illumination without relying on discrete digital state switching.
- To process analog signals through the ATmega328P 10-bit Analog-to-Digital Converter (ADC).
- To transform 10-bit input values **(0 to 1023)** into 8-bit output register values **(0 to 255)** using algorithmic scaling.
- To generate and output a stable Pulse Width Modulation (PWM) signal from Pin 9 to modulate LED intensity smoothly.

Hardware Components Used

- **Arduino Uno R3**: Microprocessor core based on the ATmega328P.
- **10 kOhm Potentiometer**: Variable resistor used as an analog voltage divider.
- **Red LED:** A standard 5 mm light-emitting diode used as the primary output actuator.
- **220 Ohm Current-Limiting Resistor**: Placed in series with the LED to prevent thermal runaway and pin overcurrent limits.
- **Half-size Breadboard**: Solderless breadboard for rapid circuit prototyping.
- **Jumper Wires**: Solid-core/stranded wire connections for signal lines and power rails.

Circuit Connections Table

Component	Pin / Terminal	Destination (Arduino / Circuit Node)	Description / Signal Type
Potentiometer	Terminal 1	Arduino 5V Pin	Positive Supply Rail (+5V DC)
Potentiometer	Wiper (Center Pin)	Arduino Analog Input Pin A0	Variable Voltage Signal Input (0 to 5V DC)
Potentiometer	Terminal 2	Arduino GND Pin	System Ground Rail (0V)
Red LED	Anode	220 Ohm Resistor (Terminal…	Anode Node
220 Ohm Resistor	Terminal B	Arduino Digital PWM Pin 9	Pulse Width Modulated Output Signal
Red LED	Cathode	Arduino GND Pin	Ground Connection

Working Principle

1. Analog-to-Digital Conversion (ADC)

The ATmega328P microcontroller features an integrated 10-bit successive-approximation ADC. When the potentiometer's wiper rotates, it varies the potential divider ratio, sweeping the input voltage on pin A0 continuously from 0V to 5V DC.

- **ADC Resolution:** 2^10 = 1024 discrete steps (0 to 1023)
- **Digital Value Formula:**

  Digital Value = (Vin / Vref) * 1023 = (Vin / 5.0V) * 1023

Thus, an analog input of 0V yields an integer value of 0, while an analog input of 5V yields an integer value of 1023.

2. Signal Mapping Function

While input sensing operates across a 10-bit scale (0 to 1023), the hardware PWM timer registers are 8-bit (0 to 255). Direct register assignment would cause signal overflow and erratic behavior. The algorithm uses linear interpolation to map the 10-bit value down to an 8-bit scale:

Output PWM = ADC * (255 / 1023) ≈ ADC / 4

In code, this is handled via the built-in map(value, 0, 1023, 0, 255) execution function.

3. Pulse Width Modulation (PWM) Output

Arduino Pin 9 leverages Timer1 to generate a high-frequency square wave (approximately 490 Hz). By varying the **Duty Cycle**—the ratio of the signal pulse "HIGH" duration to the total period—the average forward current delivered to the LED changes, effectively modulating perceived brightness.

Duty Cycle (%) = (PWM Value / 255) * 100% V_effective = V_max * (Duty Cycle / 100)

i. **PWM Value = 0**: 0% Duty Cycle → 0V effective output → LED completely OFF
ii. **PWM Value = 127**: 49.8% Duty Cycle → 薯2.5V effective output → Medium brightness
iii. **PWM Value = 255**: 100% Duty Cycle → 5V effective output → Maximum brightness

Arduino Source Code

```
C/C++
// Pin Definitions
const int potPin = A0;    // Potentiometer wiper connected to Analog Pin A0
const int ledPin = 9;     // LED connected to Digital PWM Pin 9
```

```
C/C++
void setup() {
  pinMode(ledPin, OUTPUT);  // Set LED pin as output
  Serial.begin(9600);       // Initialize serial monitor
}
```

```
C/C++
void loop() {
  int potValue = analogRead(potPin);                  // Read 10-bit ADC (0 - 1023)
  int brightness = map(potValue, 0, 1023, 0, 255);    // Map to 8-bit PWM (0 - 255)
  float dutyCycle = (brightness / 255.0) * 100.0;     // Calculate duty cycle percentage
```

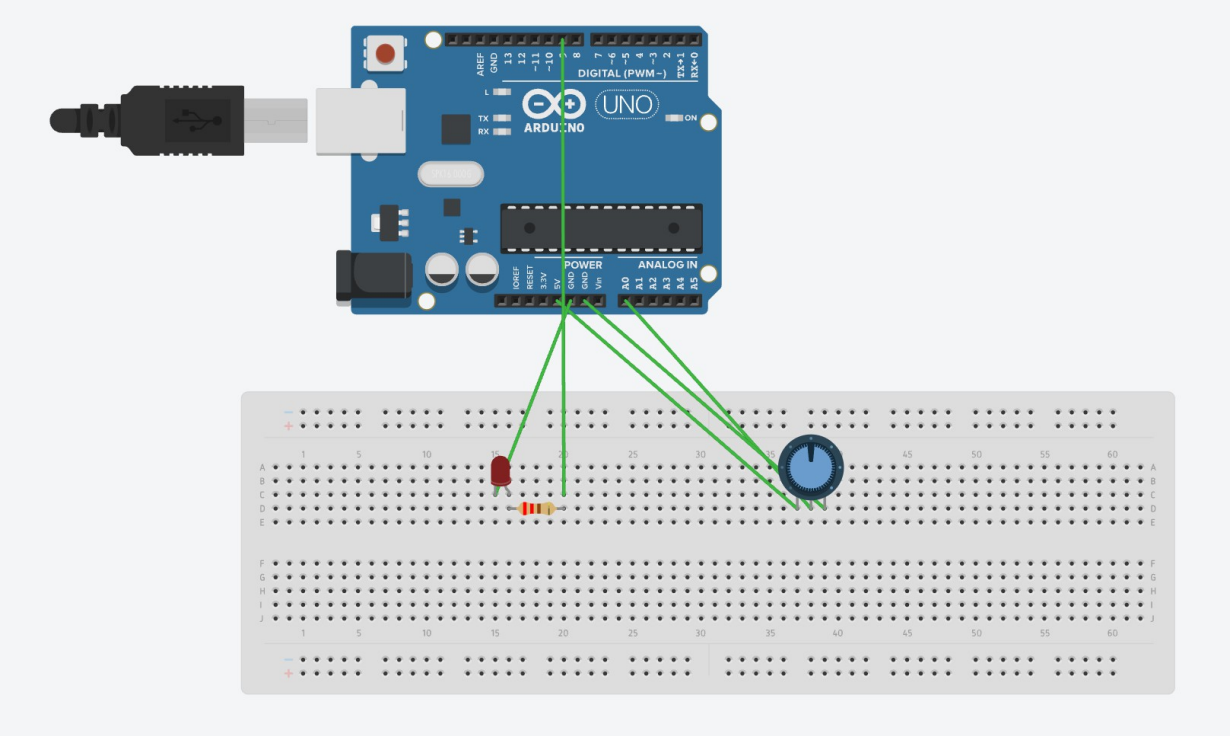
```
  analogWrite(ledPin, brightness);                    // Output PWM signal to LED

  // Telemetry output to Serial Monitor
  Serial.print("ADC Input: ");
  Serial.print(potValue);
  Serial.print(" | Mapped PWM: ");
  Serial.print(brightness);
  Serial.print(" | Duty Cycle: ");
  Serial.print(dutyCycle, 1);
  Serial.println("%");

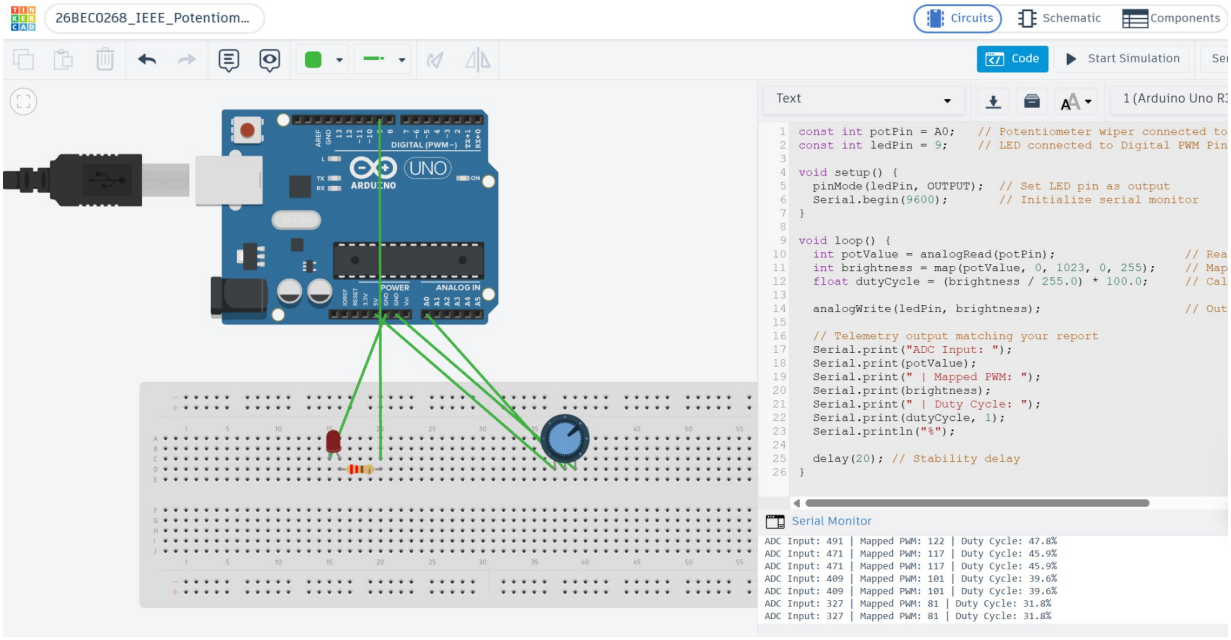
  delay(20); // Stability delay
}
```

Verification & Results

1. Full Circuit Setup

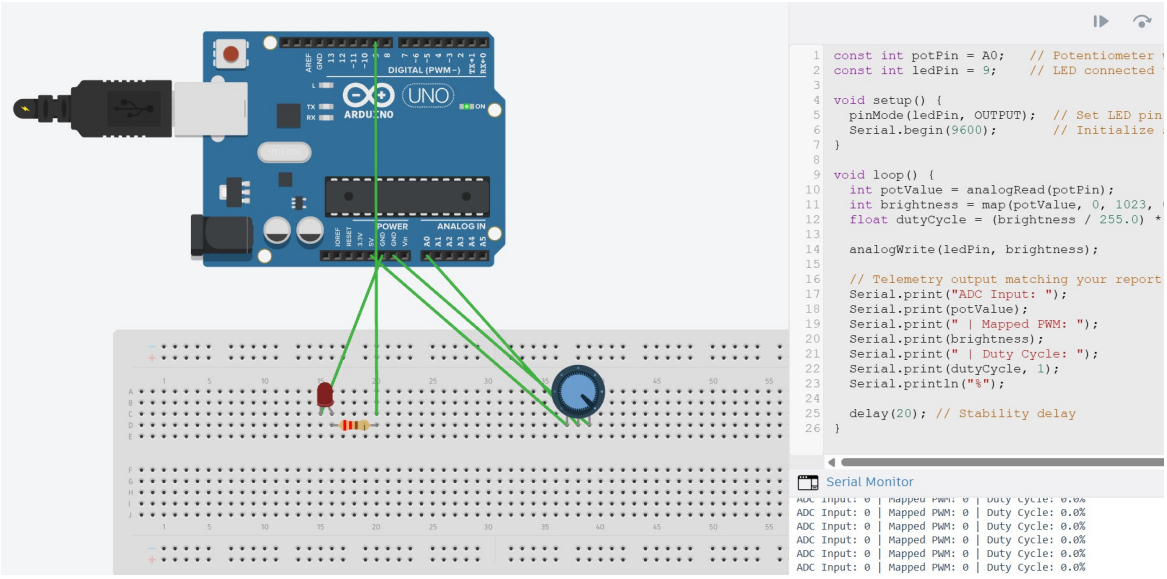


2. Serial Monitor Telemetry Output

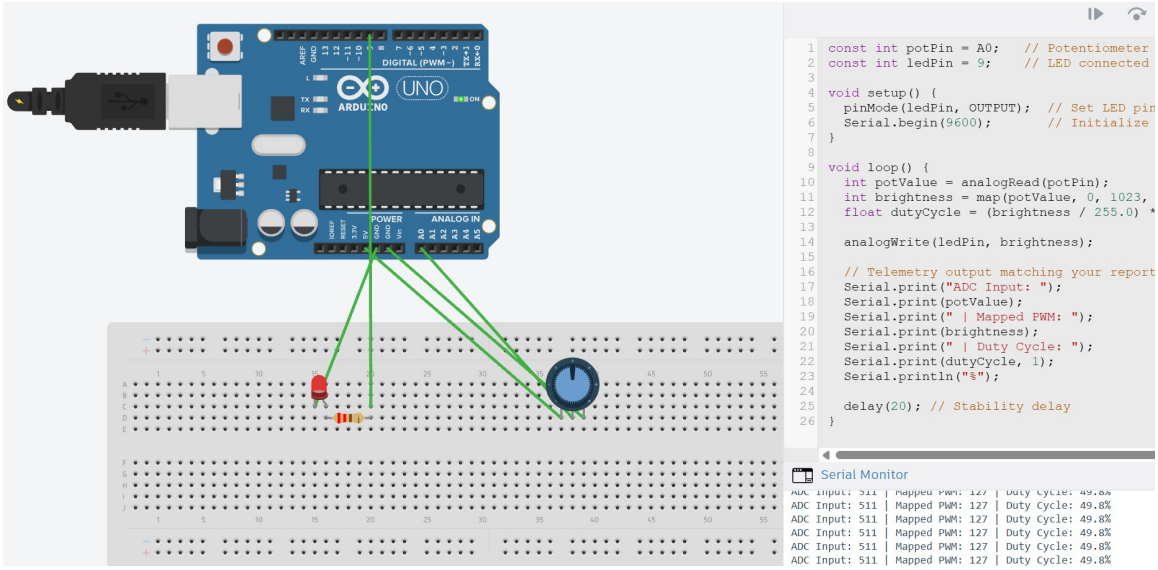


3. Output States

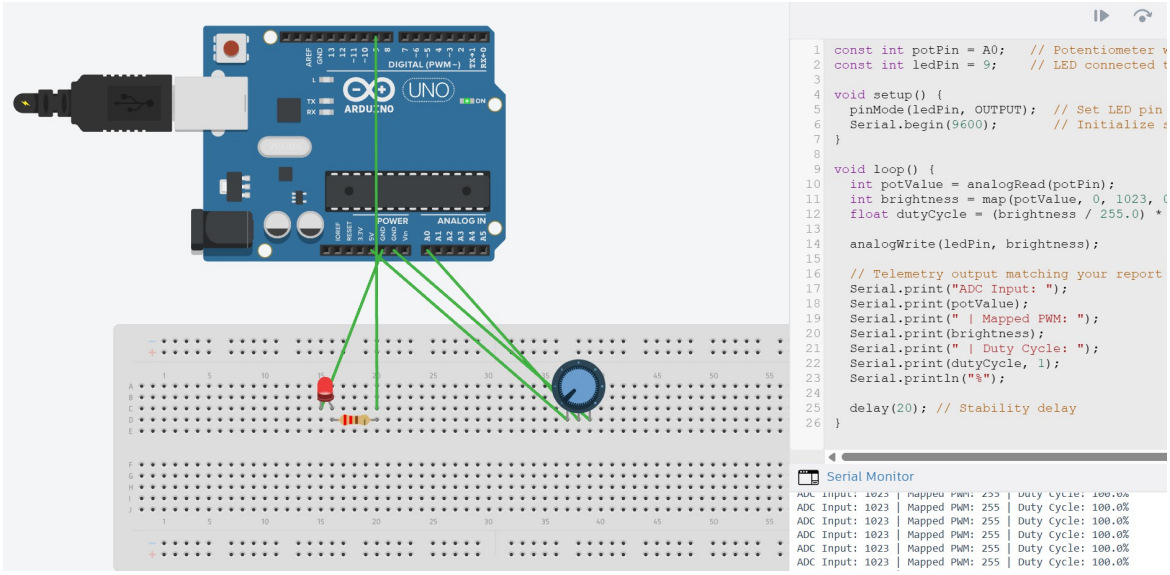
a. Low Intensity/Off State:



b. Medium Intensity State:



c. **Full Output / Maximum Brightness**:



Simulation Link Section

Access the fully configured, interactive online Tinkercad schematic and virtual testbed here:

- **Tinkercad Project File URL**: https://www.tinkercad.com/things/8gOIt6CcnVH-26bec0268ieeepotentiometer-controlled-led-brightness/editel?returnTo=%2Fthings%2F8gOIt6CcnVH-26bec0268ieeepotentiometer-controlled-led-brightness&sharecode=9vdvYcwLSjM2_8zl0NsfpldAGAaEq5rKNe0X0Nf7w eg
- **Report Verification Date**: October 29, 2026
- **Verified By**: PATNALA SUBHAOM KUMAR PATRO (26BEC0268)